

Lecture 2. Basic Classification in Data Mining

Data Mining, prof. Biljana Tojtovska Ribarski
FCSE, Skopje

Contents

1 Basic Classification in Data Mining	1
1.1 Introduction	1
1.2 Before Building a Classifier: What Is the Actual Decision Problem?	2
1.3 What Makes a Class Label Usable?	3
1.4 Running example: Medical Triage	4
1.5 General Workflow: From Training Data to Prediction	5
1.6 Main Families of Classification Methods	5
1.7 Why Decision Trees Are a Natural First Model	6
1.8 A Classical Example from the Slides	6
1.9 There Is Not Just One Tree	6
1.10 Hunt's Algorithm: The Basic Recursive Picture	7
1.11 How Test Conditions Depend on Attribute Type	7
1.12 How Continuous Attributes Are Handled	9
1.13 What Makes a Split Good?	9
1.14 Main Impurity Measures	9
1.15 Gain After Splitting	10
1.16 A Good Split Is Not Only a Numerical Split	10
1.17 Too Many Partitions and Gain Ratio	11
1.18 Prediction-Time Availability and Leakage	12
1.19 Leaves as Class Distributions, Not Just Hard Labels	12
1.20 Missing Values and Unseen Values	13
1.21 Common Modeling Mistakes Before We Even Evaluate the Classifier	14
1.22 Greedy and Local Nature of Tree Building	14
1.23 Instability of Trees	15
1.24 Axis-Parallel Limitation	16
1.25 Practical Do's and Don'ts	16
1.26 Summary	17
1.27 Questions	18

1 Basic Classification in Data Mining

1.1 Introduction

Classification is one of the most important tasks in data mining. We observe objects through a set of attributes and want to assign each object to one of several predefined classes. In many textbooks this sounds very direct: prepare a dataset, choose a classifier, train the model, and predict. But that picture is incomplete. A good classification analysis starts before any algorithm is chosen.

The first questions are conceptual:

- What exactly is the object being classified?
- What exactly does the class label mean?
- Which attributes are available when the prediction must actually be made?
- Which attributes are legitimate to use for the decision?

The main message of this lecture is that classification is not only about fitting a rule. It is about matching the rule to the meaning of the problem. A classifier may look mathematically good and still be conceptually wrong if the label is vague, the predictors are inappropriate, or the split conditions ignore the type of the data.

Definition: Classification problem

A classification problem is a supervised learning problem in which each training record is represented by a pair (x, y) , where x is a vector of attributes and y is a class label drawn from a finite set of predefined classes. The goal is to learn a model f such that

$$f(x) \in \mathcal{Y}$$

for new objects whose class is unknown.

Remark

If the classes are not predefined and the goal is instead to discover groups in the data, then the problem is not classification but clustering.

Remark

Classification should also be distinguished from regression. In classification, the output belongs to a finite set of labels, whereas in regression the output is a numerical quantity. A classification problem may be binary, multiclass, or, in some applications, multilabel. In a binary problem the model chooses between two classes; in a multiclass problem it chooses one among several classes; in a multilabel problem an object may belong to several labels at the same time. In these notes, many examples are binary because they are easier for introducing the main ideas, but the same principles of label definition, predictor legitimacy, and split logic extend beyond the two-class case.

1.2 Before Building a Classifier: What Is the Actual Decision Problem?

Before building a classifier, one should pause and ask what the real decision problem is. This is often the point where practical problems begin.

A useful way to think is:

$$\text{Objects} \rightarrow \text{Attributes} \rightarrow \text{Label} \rightarrow \text{Model} \rightarrow \text{Decision}.$$

If the first three parts are not well chosen, the rest will not fix the problem.

Definition: Classification-aware analysis

A classification-aware analysis is an analysis in which the objects, attributes, labels, and learning rule are chosen so that they match the scientific, operational, or business decision of interest.

Example: The object must be clear

Suppose a university wants to predict academic risk. Are we classifying:

- students,
- course enrollments,
- semesters,
- or student-course pairs?

These are not the same objects. A model built on *whole students* answers a question such as: “Which students are at risk of dropping out or failing overall in the next academic period?” In contrast, a model built on *student-course pairs* answers a more local question such as: “For which specific student in which specific course is there a high risk of failure or poor performance?” In this case, the object being classified is not the student alone, but the *student-course pair*. The class label may describe the expected outcome in that specific course, for example whether the student is likely to pass, fail, or be at high risk in that course. With student-course pairs across many courses, the model can learn how outcomes depend on both student characteristics and course characteristics.

Example: The decision moment must be clear

Suppose a hospital wants to classify whether a patient needs urgent intervention at triage. Then the relevant question is not “What information exists somewhere in the hospital record?” but rather “What information is actually known at the triage desk?” That difference is crucial. For example, the hospital record may later contain laboratory results, imaging findings, the specialist’s assessment, ICU admission, or the final discharge diagnosis. Although these variables may be stored in the same record, they are not suitable predictors for triage if they become known only after the triage decision has already been made.

Insight

A classifier should be designed for a specific decision moment. If the decision moment is vague, then even the best algorithm may solve the wrong problem.

1.3 What Makes a Class Label Usable?

One of the most important practical issues is the definition of the label itself. Students often treat the class label as if it were simply given. In real applications, defining the label can be one of the hardest parts of the whole task.

Example: Vague versus usable label

Consider a student dataset. A target such as

“student at risk”

is too vague unless it is defined operationally. Does it mean:

- likely to fail the current course,
- likely to fail at least three courses this semester,
- likely not to enroll next semester,
- likely to drop out within one year?

These are different classification problems and may require different predictors.

Example: Finance example

The label

“bad customer”

is vague. In contrast,

“default within 12 months after loan approval”

is operational and measurable.

Poor label definition can create several problems:

- inconsistent labels across records,
- labels that mean different things in different time periods,
- labels with measurement error,
- labels that are too broad to support a meaningful model.

Warning

A classifier does not learn the informal idea in the analyst’s head. It learns the specific label definition present in the data.

Insight

When a model behaves strangely, the problem is not always the algorithm. Sometimes the deeper problem is that the label was never sharply defined.

1.4 Running example: Medical Triage

We will use one running example throughout these notes.

Suppose a hospital wants to classify incoming patients at triage into two classes:

Urgent respiratory condition vs. Non-urgent other condition.

Each patient may be described by the variables in Table 1.

Table 1: A mixed-type medical triage example

Attribute	Type	Usually available at triage?	Comment
Age	Numeric	Yes	Measured or known immediately
Sex	Binary	Yes	Directly available
Symptom severity	Ordinal	Yes	Mild / moderate / severe
Reported symptom type	Nominal	Yes	Cough / fever / chest pain / other
CRP or lab biomarker	Continuous	Sometimes no	May arrive later than the decision moment
Patient ID	Nominal, high-cardinality	Yes	Available, but not meaningful as predictor
Final discharge diagnosis	Label or post-hoc variable	No	Must not be used as an ordinary predictor

This example is useful because it contains several types of variables at once: numeric, binary, ordinal, nominal, continuous variables that may be delayed, dangerous variables such as identifiers and post-outcome information.

1.5 General Workflow: From Training Data to Prediction

In supervised classification we observe a training set

$$\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\},$$

where each x_i is a description of an object and each y_i is its known class.

The usual workflow has two phases.

Induction phase

The learning algorithm examines the training data and constructs a model.

Application phase

The learned model is applied to unseen records to assign class labels.

Example: From training to use

Suppose past triage cases are labeled as urgent or non-urgent. A classifier is trained on those labeled cases. Later, when a new patient arrives, the model receives the patient's attributes and predicts one of the two classes.

Remark

The key point is that the model is learned from examples with known labels and then transferred to future or unseen cases.

1.6 Main Families of Classification Methods

There are few important families of classification methods:

- decision trees,
- rule-based classifiers,

- nearest-neighbor methods,
- Naïve Bayes and Bayesian belief networks,
- support vector machines,
- neural networks and deep neural networks.

There are also ensemble methods such as bagging, boosting, and random forests.

1.7 Why Decision Trees Are a Natural First Model

A decision tree represents classification as a sequence of tests.

Definition: Decision tree

A decision tree is a hierarchical classifier in which each internal node applies a test to an attribute, each branch corresponds to an outcome of that test, and each leaf gives a predicted class.

Decision trees can handle different attribute types and are easy to visualize.

Insight

A decision tree partitions the attribute space into regions and assigns one class to each region. Each root-to-leaf path is a small decision rule.

1.8 A Classical Example from the Slides

The slides illustrate decision trees through a loan default example with attributes such as HomeOwner, MaritalStatus, and AnnualIncome, and target class DefaultedBorrower. This example is useful because it shows the basic mechanics clearly:

- classification is learned from labeled examples,
- the model is then applied by following a path from the root to a leaf,
- different predictors may enter at different stages of the decision process.

This example is simplified. Real datasets usually contain mixed data types, missing values, delayed variables, noisy labels, and predictors that may or may not be legitimate.

1.9 There Is Not Just One Tree

A very important conceptual lesson is that the same training data can often be represented by more than one decision tree.

Different trees may arise because of:

- different splitting criteria,
- different tie-breaking choices,
- binary versus multi-way splits,
- different but equally reasonable local decisions.

Example: Two reasonable first questions

In the triage example, one tree may begin with symptom severity, while another begins with reported symptom type. Both may be defensible on the same training data, especially if the two candidate splits reduce impurity by similar amounts.

Warning

The learned tree is not *the* unique structure hidden in the data. It is one model produced by a particular induction procedure.

1.10 Hunt's Algorithm: The Basic Recursive Picture

A classical way to describe tree induction is Hunt's algorithm.

Definition: Records reaching a node

For a node t in a decision tree, let D_t denote the set of training records that reach that node.

The recursive logic is simple:

1. If all records in D_t belong to the same class, make t a leaf.
2. Otherwise, choose an attribute test that splits D_t into smaller subsets.
3. Create child nodes for those subsets.
4. Apply the same procedure recursively to each child.

Example: Triage version of Hunt's algorithm

Suppose the root node contains all patients. If they are not all from the same class, the algorithm might split on symptom severity. Then the subgroup of severe patients may be split again on reported symptom type, while the subgroup of mild patients may be split on age or temperature. The tree is built recursively within each subgroup.

Insight

A tree is built locally, node by node. Once a split is chosen near the top, it changes which records are available for later decisions lower in the tree.

1.11 How Test Conditions Depend on Attribute Type

One strength of decision trees is that the split condition can be matched naturally to the data type.

Binary attributes

For a binary attribute, the split is direct: one branch for each value.

Example: Binary attribute

In the triage dataset, sex is binary, so the split is simply one branch per value.

Nominal attributes

For a nominal attribute, one may use:

- a multi-way split with one branch per category, or

- a binary split that groups categories into two subsets.

Example: Nominal attribute

Reported symptom type may take values cough, fever, chest pain, and other. A tree may keep these as four separate branches, or it may group them into

$\{\text{cough, fever}\}$ vs. $\{\text{chest pain, other}\}$,

depending on the induction algorithm.

Ordinal attributes

For an ordinal attribute, the ordering should be respected. Binary splits should divide the ordered values into consecutive groups.

Example: Ordinal attribute

If symptom severity is ordered as

$\text{mild} < \text{moderate} < \text{severe}$,

then a sensible split is

$\{\text{mild}\}$ vs. $\{\text{moderate, severe}\}$,

or

$\{\text{mild, moderate}\}$ vs. $\{\text{severe}\}$.

A split such as

$\{\text{mild, severe}\}$ vs. $\{\text{moderate}\}$

ignores the order and is conceptually poor.

Warning

Ordinal variables should not be treated as if they were merely nominal. Their order is part of the information.

Continuous attributes

For a continuous attribute, one often uses a threshold test such as

$A < v$ versus $A \geq v$.

Example: Continuous attribute

In the triage example, a tree may split on CRP using a threshold such as

$\text{CRP} < 12$ vs. $\text{CRP} \geq 12$.

This does not mean that 12 is a universal medical truth. It is simply a cut that looked useful in that local part of the tree.

Remark

Thresholds in trees are model-dependent cuts. They are useful decision boundaries, not automatically scientifically fundamental thresholds.

1.12 How Continuous Attributes Are Handled

A continuous predictor may be discretized into intervals or handled directly through threshold search.

Common discretization ideas include:

- equal-width bins,
- equal-frequency bins,
- clustering-based intervals.

In decision trees, the threshold approach is often more natural because the cut can adapt to the local subset of records at a node.

Insight

A tree turns a continuous variable into a simple yes/no question: is the value smaller than this threshold or not?

1.13 What Makes a Split Good?

At a node, several tests may be possible. To compare them, trees use the idea of *purity*. A node is pure if it contains records from one class only, and impure if it contains a mixture of classes.

Definition: Node impurity

A node impurity measure quantifies how mixed the class distribution is at a node.

Common impurity measures include:

- Gini index,
- entropy,
- misclassification error.

The usual principle is:

prefer the split that creates purer child nodes.

1.14 Main Impurity Measures

Let $p_i(t)$ denote the proportion of class i at node t and let c be the total number of classes.

Gini index

$$\text{Gini}(t) = 1 - \sum_{i=1}^c p_i(t)^2.$$

Entropy

$$\text{Entropy}(t) = - \sum_{i=1}^c p_i(t) \log_2 p_i(t).$$

Misclassification error

$$\text{Error}(t) = 1 - \max_i p_i(t).$$

Remark

Gini and entropy are usually more sensitive than misclassification error when comparing candidate splits.

1.15 Gain After Splitting

Suppose a parent node P is split into child nodes $v_1, \dots, v_k$. If N is the number of records at the parent and N_{v_j} is the number of records at child v_j , then the weighted post-split impurity is

$$M = \sum_{j=1}^k \frac{N_{v_j}}{N} I(v_j),$$

where I is the chosen impurity measure.

The gain is then

$$\text{Gain} = I(P) - M.$$

If entropy is used, this reduction is called *information gain*.

Example: A Gini example

Suppose a node contains 12 patients: 7 urgent and 5 non-urgent. Then

$$\text{Gini}(P) = 1 - \left(\frac{7}{12}\right)^2 - \left(\frac{5}{12}\right)^2 = 0.486.$$

Assume a split creates two child nodes with class counts (5, 1) and (2, 4). Then

$$\text{Gini}(N_1) = 0.278, \quad \text{Gini}(N_2) = 0.444.$$

So the weighted impurity is

$$M = \frac{6}{12} \cdot 0.278 + \frac{6}{12} \cdot 0.444 = 0.361,$$

and the gain is

$$0.486 - 0.361 = 0.125.$$

The split is preferred because it makes the child nodes more class-homogeneous than the parent.

1.16 A Good Split Is Not Only a Numerical Split

Students often learn the following pattern:

$$\text{impurity goes down} \Rightarrow \text{the split is good}.$$

That is useful, but incomplete. A good split is not only a numerical split. It should also make sense for the problem.

Example: A misleading split

Suppose the triage dataset contains PatientID. Since every patient has a different ID, splitting on PatientID could create many tiny nodes, each containing one patient. These nodes are perfectly pure, so the split may look extremely attractive mathematically. But it is meaningless. PatientID does not contain meaningful predictive structure, it is arbitrary identifier. The tree is memorizing the training cases, which leads to severe overfitting and

does not generalize to new patients with unseen IDs.

Example: A more meaningful split

In the same dataset, a split on symptom severity may separate many urgent cases from non-urgent ones. This split is not only numerically useful; it also reflects a medically meaningful distinction.

Example: A disguised target variable

Suppose a variable called `PlacedInEmergencyWard` is present in the data. It may reduce impurity dramatically because it was recorded after clinicians had already recognized the urgent cases. As a predictor, it is inappropriate because it is too close to the target itself. The model is not really learning from patient characteristics; it is learning from a variable that was created because someone had already partly recognized the target outcome (urgent case).

Insight

A strong split should be judged in two ways:

1. Does it reduce impurity?
2. Does it represent a meaningful distinction that could legitimately be used at prediction time?

1.17 Too Many Partitions and Gain Ratio

A classical problem is that some splitting criteria may favor tests that create many very small, very pure child nodes. This may happen with attributes such as:

- CustomerID,
- StudentID,
- invoice number,
- exact timestamp,
- product code with many rare values.

Warning

A split that memorizes the training data is usually not a meaningful split in classification, because it does not capture a pattern that can generalize to new cases. It may only be useful in special identity-based settings, where the same entities are expected to reappear and the task is closer to lookup than to general prediction.

Gain ratio was introduced to reduce this problem. It adjusts information gain by the entropy of the partition itself:

$$\text{GainRatio} = \frac{\text{Gain}_{\text{split}}}{\text{SplitInfo}}, \quad \text{SplitInfo} = - \sum_{i=1}^k \frac{n_i}{n} \log_2 \frac{n_i}{n}.$$

Remark

Gain ratio discourages splits that fragment the data excessively into many tiny groups.

1.18 Prediction-Time Availability and Leakage

One of the most important practical questions in classification is whether a predictor is available at the moment the prediction must be made.

Example: Credit-risk leakage

If the task is to predict *at loan approval time* whether an applicant will later default, then variables such as the number of missed payments after loan approval are not legitimate predictors, because they are not yet known at the moment of decision. However, if the task is to predict default *after the loan is already active*, then such variables may be entirely appropriate, because the classification problem is now a different one.

Example: Medical leakage

Suppose the task is to classify patients at triage. Then a laboratory value that becomes available three hours later, or a diagnosis confirmed after imaging or surgery, should not be treated as an ordinary predictor for the triage-time decision.

Example: A subtle leakage example

Suppose a model is trained to predict whether a patient has severe pneumonia, and one predictor is whether the patient was immediately sent for chest CT by the specialist. This may look like a normal hospital variable, but in reality it partly reflects a clinician's judgment made after seeing the patient. The model is then learning from hidden downstream decisions, not only from raw patient information.

If the prediction is supposed to be made before the specialist decides on chest CT, then this variable should not be used, because it leaks later clinical judgment into the model. However, if the classification task is explicitly redefined for a later stage of care, when that decision is already known, then the variable may be legitimate, but the model is now solving a different problem.

Warning

A variable can be present in the dataset and still be conceptually illegal for prediction.

Insight

When building a classifier, always imagine the exact moment of prediction. If the variable would not yet be known then, it should not be used as an ordinary predictor.

1.19 Leaves as Class Distributions, Not Just Hard Labels

In practice, a leaf corresponds to a local class distribution.

Table 2: Two leaves with the same predicted class but different support

Leaf	Urgent	Non-urgent	Majority class	Total	Interpretation
A	18	2	Urgent	20	Strong evidence for urgent
B	6	5	Urgent	11	Much weaker support

Both leaves in Table 2 predict the class Urgent, but they are not equally convincing.

Example: Same class, different support

Suppose a new patient reaches Leaf A. The training cases in that leaf were 18 urgent and 2 non-urgent. Now suppose a different patient reaches Leaf B, where the counts were 6 urgent and 5 non-urgent. Both patients receive the same class label, but the underlying local evidence is much weaker in Leaf B.

Remark

A leaf tells us not only which class is predicted, but also how the training data were distributed there. In many implementations, those counts are used to estimate local class probabilities.

Two records may receive the same predicted class and yet arrive there with very different degrees of support. Therefore, a leaf should be read not only as a final label, but also as a summary of the local evidence contained in the training data. This becomes important later when we discuss evaluation and decision making, because the practical usefulness of a classification rule depends not only on which class is predicted, but also on how strongly the data support that prediction in the relevant region of the attribute space.

Insight

The majority class gives the decision, while the class composition at the leaf tells us how clear or ambiguous that local decision was.

This observation will matter later when we discuss confidence, errors, and evaluation measures.

1.20 Missing Values and Unseen Values

Real datasets are rarely perfectly complete.

Two practical issues arise very often:

1. some training or test records have missing values;
2. a test record may contain a category not seen during training.

Missing values

A missing value does not automatically mean zero, absence, or the average. It may mean:

- the measurement was not collected,
- the measurement failed,
- the variable was not applicable,
- or the missingness itself may carry information.

Example: Why missingness may matter?

In the triage example, suppose CRP is missing mainly for patients who were judged so clearly non-urgent that no laboratory test was ordered. Then “CRP missing” is not random. It reflects part of the clinical process.

Unseen categories

A test record may contain a category not present in the training data.

Example: Unseen category

Suppose the training data contain reported symptom types

$\{\text{cough, fever, chest pain, other}\},$

but later a new triage form introduces the category **loss of smell**. A tree built on the earlier data must still have some policy for handling this unseen value.

Warning

Missing values and unseen categories are not strange corner cases. They are part of normal deployment reality.

Remark

Classification system must define how such situations are handled, they should not be ignored silently.

1.21 Common Modeling Mistakes Before We Even Evaluate the Classifier

Several things can go wrong long before we reach training error, test error, or pruning.

1. **Vague target label.** Example: “bad customer” instead of “default within 12 months.”
2. **Leakage.** Example: using missed payments recorded after loan approval to predict default.
3. **Identifiers as predictors.** Example: PatientID or StudentID creating artificially pure child nodes.
4. **Ignoring the meaning of missingness.** Example: treating missing CRP exactly like a measured low CRP.
5. **Ignoring unseen categories.** Example: a new symptom category appearing after deployment.
6. **Destroying ordinal structure.** Example: treating symptom severity as three unrelated categories (there is a natural order).
7. **Overinterpreting thresholds.** Example: reading a learned cut such as $\text{Age} < 63$ as a universal scientific law.
8. **Using predictors too close to the target.** Example: a downstream hospital placement variable that almost directly reveals urgency.
9. **Believing that the most impurity-reducing split is automatically meaningful.** Example: a split on exact timestamp during a temporary outbreak wave.

Insight

Good classification is not only about choosing a method. It is about avoiding conceptually wrong formulations of the problem.

1.22 Greedy and Local Nature of Tree Building

Decision-tree induction is usually greedy. At each node, the algorithm chooses the split that looks best at that moment according to the chosen impurity criterion.

This is practical, but it has a limitation.

Warning

A split that is locally best need not lead to the globally best tree.

Example

Consider two binary predictors A and B and the class in Table 3.

Table 3: An example showing local difficulty

A	B	Class
0	0	No
0	1	Yes
1	0	Yes
1	1	No

Neither A nor B alone gives a very helpful first split at the root, because each value of A and each value of B still contains a mixed class distribution. Yet together they determine the class perfectly. This shows that a tree may miss structure when only local one-step improvements are considered.

Remark

This does not make trees useless. It simply means that their construction is heuristic and local rather than globally optimal.

Remark

For the XOR-type pattern above, the combined predictor (A, B) determines the class perfectly, even though neither A nor B alone is very informative at the root. This shows that the predictive structure may lie in the interaction between variables rather than in their separate effects.

We can detect such an interaction by observing that the class remains mixed when we examine A or B separately, but becomes much clearer when we examine their joint values. In practice, this may be done by feature engineering, for example by creating a combined predictor such as (A, B) , or by using methods that can model joint structure more directly, such as multivariate tree methods, kernel methods, or neural networks.

1.23 Instability of Trees

Decision trees are interpretable, but they can also be unstable.

A small change in the dataset may lead to:

- a different root split,
- different thresholds,
- a different overall tree structure.

Example: A small change, a different first question

Suppose in the triage data the two best root candidates are symptom severity and age, with nearly identical impurity reduction. If one borderline patient is added, removed, or relabeled, the preferred root split may change from severity to age. The whole downstream tree may then change as well.

Warning

Interpretability should not be confused with robustness. A tree can be easy to read and still be sensitive to small changes in the data.

1.24 Axis-Parallel Limitation

A standard decision tree usually splits on one attribute at a time:

$$x_j < c.$$

This means the decision boundaries are axis-parallel.

This has clear advantages:

- the model is easy to explain,
- the tests are simple,
- mixed data are handled naturally.

But it also has a limitation:

- some class boundaries are diagonal, curved, or interaction-based, and cannot be represented compactly with only one-variable splits.

Example: A diagonal rule

Suppose the class depends roughly on whether

$$x_1 + x_2 > c.$$

A tree with only axis-parallel splits may need several cuts to approximate this decision boundary, even though the real rule is simple when written in terms of the combination $x_1 + x_2$.

Remark

This is not a reason to reject trees. It is a reason to understand what type of geometric decision structure they represent naturally.

1.25 Practical Do's and Don'ts

Do's

- Define the object, attributes, and class label precisely.
- Check that predictors are available at the moment of prediction.
- Respect attribute type when defining split conditions.
- Preserve order when splitting ordinal variables.

- Ask whether a split is conceptually meaningful, not only numerically attractive.
- Interpret a leaf as a local class distribution, not only as a hard label.
- Be suspicious of identifiers and near-identifiers.
- Ask what missing values mean before deciding how to handle them.
- Remember that unseen categories can occur in practice.
- Treat a learned tree as one possible model, not as the unique truth.

Don'ts

- Do not think that a classification problem is well defined just because a dataset has a column that could be used as a label.
- Do not assume that every variable stored in the dataset is automatically a legitimate predictor.
- Do not believe that a split is useful only because it produces very pure child nodes on the training data.
- Do not interpret a learned threshold as if it were automatically a scientific or domain-specific law.
- Do not treat a tree as if it were the unique or final explanation hidden in the data.
- Do not forget that the same variable may be useful in one prediction setting and inappropriate in another, depending on the decision moment.
- Do not read a leaf label as if it expressed absolute certainty; always remember that it comes from the local class composition of the training data.
- Do not assume that missing values are harmless technical nuisances; they may reflect part of the real process that generated the data.
- Do not expect a one-variable-at-a-time split to capture every important interaction present in the data.
- Do not judge a classifier only by how easy it is to read; interpretability and reliability are not the same thing.

1.26 Summary

Classification is the supervised task of learning a mapping from attributes to predefined class labels. But a good classification analysis begins before the model is trained. It begins with a careful definition of the object, the label, the decision moment, and the set of predictors that are genuinely available.

Decision trees are an especially useful first model because they represent classification as a sequence of explicit tests. They make it easy to see how induction works, how a record is classified, how different attribute types lead to different split conditions, and how impurity measures guide the choice of splits.

The core material remains essential: the distinction between training and prediction, the recursive idea of Hunt's algorithm, the role of binary, nominal, ordinal, and continuous attributes, and the use of Gini, entropy, information gain, and gain ratio to compare candidate splits.

At the same time, real classification practice requires more than formulas. A classifier only learns the label that has been defined. Predictors must be available at prediction time. Not every impurity-reducing split is meaningful. Leaves store local class evidence, not only class names. Missing values and unseen categories are normal practical issues. Trees are interpretable but can be unstable, and their standard split geometry is axis-parallel.

Thus, the real lesson of basic classification is not only how to build a tree. It is how to think correctly about what the label means, what information is legitimate, what a split actually says, and when a mathematically attractive classifier may still be the wrong one for the problem.

1.27 Questions

1. Define a classification problem formally. What are the roles of x and y , and how does classification differ from clustering?
2. Why is label definition one of the most important parts of a classification problem? Give one example of a vague label and rewrite it into a more operational one.
3. Explain why the decision moment matters in classification. What is leakage, and why can the same variable be legitimate in one prediction setting but inappropriate in another?
4. Consider an online shopping platform that wants to classify whether a transaction is **fraudulent** or **legitimate**. Suppose the available variables are: purchase amount, payment method, delivery speed, customer region, and whether the card was blocked two days later. Classify these predictors as numeric, nominal, ordinal, or binary, and explain which of them would be inappropriate if the prediction must be made at the moment the transaction is placed.
5. The same platform now wants to classify not a single transaction, but whether a *customer account* is **high-risk** or **low-risk** for fraud in the next 30 days. Give examples of predictors that would be more suitable for this account-level classification problem than for the transaction-level problem above, and explain why the object being classified has changed.
6. What is node impurity in a decision tree? State the formulas for the Gini index, entropy, and misclassification error.
7. Why can a split on **PatientID** or another identifier look very good numerically and still be a bad split? What does gain ratio try to correct in such situations?
8. Why is a leaf better understood as a local class distribution rather than only as a hard class label? Illustrate your answer by comparing two leaves that predict the same class but have different class counts.
9. Consider the following training set with predictors A , B , C and class label **Class**:

A	B	C	Class
1	0	0	No
1	0	0	No
1	0	1	Yes
0	0	0	No
0	0	1	No
0	1	0	Yes
0	1	1	Yes
1	1	0	Yes

Using the Gini measure, compute the impurity of the root node and the weighted Gini index obtained by splitting on A , on B , and on C . Choose the best root split and continue building the decision tree, again using the Gini measure at each non-pure node, until all leaves are pure. Finally, show how a new object with values $(A = 1, B = 1, C = 1)$ would be classified by your tree.

10. The following example shows that neither A nor B alone is very informative, while their combination is:

A	B	Class
0	0	No
0	1	Yes
1	0	Yes
1	1	No

What does this example show about greedy one-variable-at-a-time tree induction? How might interaction features or multivariate methods help in such a case?